

---

# Homeostatic Token Saturation for Text Generation

---

**Giovanna Oliveira Martins**  
Universidade Federal de Uberlândia  
gimartins@ufu.br

## Abstract

Repetition remains a persistent challenge in autoregressive text generation, often reducing lexical diversity despite the use of stochastic decoding strategies. Existing repetition control methods, such as Presence and Frequency Penalties, rely exclusively on cumulative token statistics and ignore when previous occurrences took place. This paper introduces Homeostatic Token Saturation, a decoding strategy that associates each vocabulary token with a dynamic saturation state. Saturation increases whenever a token is generated and gradually decays over time, discouraging recently repeated tokens while enabling previously penalized tokens to recover naturally. Experiments conducted on a decoder-only Transformer trained on the Tiny Shakespeare corpus compare the proposed method with Top- $k$  sampling, Presence Penalty, and Frequency Penalty. Results show that the proposed approach provides consistent but moderate improvements in lexical diversity and repetition over standard Top- $k$  decoding while avoiding the substantial increase in perplexity observed for Frequency Penalty.

## 1 Introduction

Large language models (LLMs) based on the Transformer architecture have achieved remarkable performance in natural language generation. These models generate text through autoregressive decoding, where each token is generated conditioned on the previously generated context. Although this formulation provides a flexible framework for text generation, the decoding process strongly depends on the probability distribution learned during training, which may lead to repetitive and less diverse outputs [3, 4].

A central challenge in neural text generation is therefore controlling the trade-off between coherence and diversity. Sampling strategies such as Top- $k$  and nucleus sampling modify the decoding distribution to avoid unlikely continuations while preserving generation variability [2, 3]. However, these approaches primarily operate by truncating or reshaping the probability distribution and do not explicitly model the repeated activation of individual tokens during generation.

Repetition mitigation methods have been proposed to address this limitation by directly modifying token probabilities. Presence and frequency penalties reduce the likelihood of previously generated tokens by considering whether a token has appeared or how frequently it has occurred in the generated sequence [1]. Nevertheless, these approaches rely on cumulative statistics and do not incorporate temporal information. As a result, tokens remain penalized based solely on their historical frequency, regardless of whether their previous occurrence was recent or occurred many decoding steps earlier.

This work introduces Homeostatic Token Saturation (HTS), a dynamic repetition control mechanism inspired by the concept of homeostasis in biological systems. Instead of maintaining only cumulative token counts, the proposed approach associates each vocabulary token with a continuously updated saturation state. This state increases when a token is generated and gradually decays over time, discouraging recently repeated tokens while enabling previously suppressed tokens to recover their probability mass.

We evaluate the proposed method on the Tiny Shakespeare benchmark using a decoder-only Transformer model trained from scratch. The evaluation compares the proposed strategy against Top- $k$  sampling, Presence Penalty, and Frequency Penalty using diversity, repetition, and fluency metrics.

## 2 Homeostatic Token Saturation

The proposed method draws inspiration from biological homeostasis. In living organisms, repeated stimulation temporarily alters the internal state of physiological systems, but this alteration gradually dissipates, restoring equilibrium over time. Analogously, each generated token temporarily increases its saturation level, which gradually decays back toward zero, restoring its sampling probability.

Let  $V$  denote the vocabulary and let  $s_i^{(t)}$  represent the saturation state associated with token  $i$  at decoding step  $t$ . Initially,

$$s_i^{(0)} = 0, \quad \forall i \in V.$$

Before generating each new token, the saturation state undergoes an exponential decay,

$$s_i^{(t)} = \lambda s_i^{(t-1)},$$

where  $\lambda \in [0, 1]$  controls the recovery rate. Values closer to one result in slower recovery, whereas smaller values rapidly forget previous activations.

After selecting the next token  $k$ , only its saturation state is reinforced,

$$s_k^{(t)} \leftarrow s_k^{(t)} + \delta,$$

where  $\delta > 0$  determines the saturation increment produced by each occurrence.

The saturation state is then used to modify the decoding logits. Given the original logits  $z_i$ , the adjusted logits become

$$\tilde{z}_i = z_i - \gamma s_i^{(t)},$$

where  $\gamma > 0$  controls the influence of saturation on the sampling distribution.

Unlike frequency based penalties, the proposed formulation depends not only on how often a token has appeared but also on when it appeared. Frequently repeated tokens receive stronger penalties immediately after generation, while their probabilities gradually recover through the decay process. Consequently, the mechanism encourages lexical diversity without permanently suppressing words that may become contextually appropriate later in the sequence.

## 3 Experimental Setup

### 3.1 Experimental Design

All experiments were conducted using a decoder-only Transformer implemented with the Hugging-Face GPT2LMHeadModel framework. The model was configured with four Transformer decoder blocks, four attention heads, model dimension of 256, context length of 128 tokens, dropout rate of 0.1, and batch size of 32. Model parameters were optimized using the AdamW optimizer with a learning rate of  $3 \times 10^{-4}$  for up to 50 training epochs, employing early stopping with a patience of five epochs.

The model was trained from scratch on the Tiny Shakespeare corpus, a widely adopted benchmark for language modeling. The dataset was partitioned into training (90%), validation (5%), and test (5%) subsets. Model parameters were learned exclusively from the training set, while the validation set was used for model selection based on the lowest validation loss. The test set remained completely unseen during training and was used solely to construct the evaluation prompts.

Tokenization was performed using the GPT-2 Byte Pair Encoding (BPE) tokenizer, resulting in a vocabulary of 50,257 tokens. During evaluation, 100 prompts of 32 tokens each were randomly extracted from the test partition. For every prompt, each decoding strategy generated three continuations of up to 512 new tokens using Top- $k$  sampling with  $k = 50$ , each with a different random seed. All methods employed the same trained language model, prompt set, seeds, and decoding configuration, ensuring that the decoding strategy was the only experimental variable.

The repetition control strategies require additional method specific hyperparameters. Presence Penalty used  $\alpha = 0.5$ , Frequency Penalty used  $\beta = 0.5$ , and the proposed HTS employed  $\gamma = 0.5$ , exponential decay factor  $\lambda = 0.95$ , and saturation increment  $\delta = 1.0$ . These values were selected empirically, without tuning specifically for any individual baseline, to represent an intermediate penalty strength while maintaining a uniform comparison across all methods.

All experiments were implemented in Python 3.12.7 using a Jupyter Notebook environment. Training and evaluation were performed on a Windows 11 computer with an Intel Core Ultra 7 processor and 32 GB of RAM, using only the CPU.

### 3.2 Baselines

The proposed HTS was compared against three decoding strategies commonly adopted in autoregressive language models.

- **Top- $k$  Sampling:** standard Top- $k$  sampling without any repetition penalty [2].
- **Presence Penalty:** applies a fixed penalty to tokens that have already appeared in the generated sequence. The adjusted logits are computed as

$$\tilde{z}_i = z_i - \alpha \mathbf{1}(c_i > 0),$$

where  $c_i$  denotes the number of previous occurrences of token  $i$ ,  $\mathbf{1}(\cdot)$  is the indicator function, and  $\alpha > 0$  controls the penalty strength.

- **Frequency Penalty:** penalizes tokens proportionally to their cumulative frequency in the generated sequence. The adjusted logits are computed as

$$\tilde{z}_i = z_i - \beta c_i,$$

where  $\beta > 0$  controls how strongly repeated tokens are penalized.

- **HTS (proposed):** dynamically adjusts token probabilities according to the saturation model described in Section 2.

### 3.3 Evaluation Metrics

The evaluation aims to characterize the trade-off between lexical diversity, repetition reduction, and fluency. Lexical diversity was measured using Distinct-1 and Distinct-2, which compute the proportion of unique unigrams and bigrams, respectively. Repetition was quantified through unigram and bigram repetition rates, while vocabulary richness was assessed using Shannon entropy.

Finally, fluency was assessed using perplexity, computed by evaluating the generated sequences under the same trained language model used for generation. As an intrinsic evaluation metric, perplexity measures how well the generated sequences conform to the probability distribution learned during training, with lower values indicating that the generated text is considered more probable by the model.

Statistical significance was assessed using paired hypothesis tests. The Shapiro-Wilk test was first applied to verify normality of paired differences. When normality was satisfied, a paired Student’s  $t$ -test was employed; otherwise, the Wilcoxon signed-rank test was adopted.

## 4 Results and Discussion

Table 1 reports the average performance over the 300 generated continuations. Overall, stronger repetition penalties tend to increase lexical diversity at the expense of higher perplexity.

Table 1: Mean performance over 300 generated continuations. Higher is better for Distinct- $n$  and Entropy, while lower is better for Repetition and Perplexity.

| Method    | D1           | D2           | UniRep       | BiRep        | Ent.         | PPL          |
|-----------|--------------|--------------|--------------|--------------|--------------|--------------|
| Baseline  | 0.379        | 0.767        | 0.621        | 0.233        | 6.598        | <b>20.41</b> |
| Presence  | 0.433        | 0.812        | 0.567        | 0.188        | 6.823        | 22.79        |
| Frequency | <b>0.506</b> | <b>0.955</b> | <b>0.494</b> | <b>0.045</b> | <b>7.661</b> | 115.27       |
| HTS       | 0.390        | 0.799        | 0.610        | 0.201        | 6.743        | 23.36        |

Baseline Top- $k$  sampling achieved the lowest perplexity, indicating the closest adherence to the learned distribution, but also exhibited the highest unigram and bigram repetition rates and the lowest diversity among repetition control methods. Presence Penalty partially alleviated this issue, increasing Distinct-1, Distinct-2, and entropy while reducing repetition with only a modest increase in perplexity, suggesting that a fixed penalty already discourages immediate token reuse.

Frequency Penalty achieved the highest lexical diversity, obtaining the best Distinct-1, Distinct-2, Shannon entropy, and repetition scores. However, its cumulative penalty also produced the highest perplexity, indicating that the stronger suppression of repeated tokens moves generation farther from the language model distribution.

HTS exhibited behavior comparable to Presence Penalty, while remaining substantially less aggressive than Frequency Penalty. Relative to the Baseline, it improved all diversity and repetition metrics while increasing perplexity only moderately. Compared with Presence Penalty, it produced slightly lower lexical diversity, while no statistically significant difference was observed in perplexity ( $p = 0.084$ ).

Overall, the results indicate that temporal saturation produces a conservative repetition control mechanism. Rather than maximizing lexical diversity, it offers a compromise between diversity improvements and preservation of the language model distribution.

## 5 Conclusion and Limitations

This work proposed Homeostatic Token Saturation, a repetition control mechanism that introduces a dynamic saturation state into autoregressive decoding. Unlike cumulative repetition penalties, the proposed approach incorporates temporal information through exponential recovery, allowing token probabilities to recover naturally over time. Experimental results showed that the method effectively reduces repetition and increases lexical diversity relative to standard Top- $k$  sampling while preserving substantially lower perplexity than Frequency Penalty, yielding a balanced trade-off between diversity and fluency.

This study also presents several limitations. The experiments were conducted on a relatively small language model trained on the Tiny Shakespeare corpus, and the hyperparameters of all repetition control strategies were selected empirically rather than through systematic optimization. Future work includes evaluation on larger pretrained language models and systematic hyperparameter optimization.

## References

- [1] Chetan Arora, Ahnaf Ibn Sayeed, Sherlock Licorish, Fanyu Wang, and Christoph Treude. Optimizing llms for code generation: Which hyperparameter settings yield the best results? In *APSEC*, 2024.
- [2] Angela Fan, Mike Lewis, and Yann Dauphin. Hierarchical neural story generation. In *ACL*, 2018.
- [3] Ari Holtzman et al. The curious case of neural text degeneration. In *ICLR*, 2020.
- [4] Clara Meister et al. Typical decoding for natural language generation. *Transactions of the Association for Computational Linguistics*, 2023.